

TRAVAUX PRATIQUES

Apache Kafka

Architecture, Déploiement et Programmation

Déploiement mono-machine — Sans conteneurs

Sommaire

Ce TP est organisé en six parties progressives. Chaque partie comporte des exercices à réaliser et des questions de réflexion.

- Partie 1 : Introduction et préparation de l'environnement
- Partie 2 : Installation de Kafka et premier démarrage
- Partie 3 : Manipulation en ligne de commande (CLI)
- Partie 4 : Programmation Producer / Consumer en Java
- Partie 5 : Concepts avancés — partitions, groupes de consommateurs, offsets
- Partie 6 : Mini-projet — Pipeline de logs en temps réel
- Annexes : Dépannage, références, grille d'évaluation

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant sera capable de :

- Installer et démarrer un cluster Kafka mono-nœud sans Docker
- Comprendre les concepts de topic, partition, offset, groupe de consommateurs
- Produire et consommer des messages via la CLI et via l'API Java
- Diagnostiquer les problèmes courants (lag, rebalance, broker down)
- Concevoir un pipeline de données simple basé sur Kafka

Partie 1 — Introduction et préparation de l'environnement

1.1 Qu'est-ce qu'Apache Kafka ?

Apache Kafka est une plateforme distribuée de streaming d'événements, conçue à l'origine par LinkedIn et donnée à la fondation Apache en 2011. Contrairement aux files de messages classiques (RabbitMQ, ActiveMQ), Kafka conserve les messages dans un journal (log) immuable et persistant : chaque consommateur lit les messages à son propre rythme, sans les détruire.

Cette caractéristique fait de Kafka l'épine dorsale de nombreuses architectures modernes : ingestion de logs, event sourcing, microservices événementiels, pipelines de données pour l'analytique en temps réel, et plus récemment Change Data Capture (CDC) avec des outils comme Debezium.

Concepts fondamentaux à mémoriser

Concept	Définition
Broker	Un serveur Kafka. Un cluster est composé d'un ou plusieurs brokers.
Topic	Catégorie nommée à laquelle les producteurs envoient des messages.
Partition	Sous-division ordonnée d'un topic. Unité de parallélisme.
Offset	Identifiant numérique unique d'un message à l'intérieur d'une partition.
Producer	Application cliente qui publie des messages dans un topic.
Consumer	Application cliente qui lit des messages depuis un topic.
Consumer Group	Ensemble de consommateurs qui se partagent la lecture d'un topic.
Replication Factor	Nombre de copies d'une partition (pour la tolérance aux pannes).
ZooKeeper / KRaft	Service de coordination du cluster (KRaft remplace ZooKeeper depuis Kafka 3.x).

1.2 Architecture cible du TP

Pour ce TP, chaque étudiant déploie sur sa propre machine un cluster Kafka constitué d'un seul broker, fonctionnant en mode KRaft (sans ZooKeeper). Cette architecture suffit pour apprendre tous les concepts de la CLI et de l'API. Les questions de réplication multi-broker sont abordées en théorie.

Pourquoi KRaft et pas ZooKeeper ?

Depuis Kafka 3.3, le mode KRaft est marqué « production ready ». Depuis Kafka 4.0, ZooKeeper est complètement supprimé. KRaft est plus simple à déployer (un seul processus à gérer), démarre plus vite, et sera la seule option dans le futur. Nous l'utilisons donc dès le départ.

1.3 Prérequis matériels et logiciels

Configuration minimale recommandée

Ressource	Minimum	Recommandé
RAM	4 Go	8 Go ou plus
Disque libre	5 Go	10 Go ou plus
CPU	Dual-core	Quad-core
OS	Linux / macOS / Windows 10+	Linux ou macOS

Logiciels à installer

- Java JDK 17 ou supérieur (Kafka 3.7+ impose Java 11 minimum, Kafka 4.x impose Java 17)
- Un terminal (PowerShell sous Windows, Bash/Zsh sous Linux/macOS)
- Un éditeur de code (VS Code, IntelliJ IDEA, Sublime Text...)
- Maven 3.8+ (pour la partie programmation Java)
- Git (pour récupérer les exemples de code)

1.4 Vérification de Java

Avant toute installation, vérifiez la présence de Java :

```
# Sous Linux / macOS / Windows (PowerShell)
java -version
javac -version

# Sortie attendue (la version peut varier mais doit être >= 17)
# openjdk version "17.0.10" 2024-01-16
# OpenJDK Runtime Environment ...
```

Si Java n'est pas installé

- Linux (Debian/Ubuntu) : `sudo apt update && sudo apt install -y openjdk-17-jdk`
- Linux (Fedora/RHEL) : `sudo dnf install -y java-17-openjdk-devel`
- macOS : `brew install openjdk@17`
- Windows : téléchargez le MSI depuis adoptium.net (Eclipse Temurin 17)

1.5 Variables d'environnement

Nous allons définir deux variables d'environnement utilisées dans tout le TP :

- `KAFKA_HOME` : le répertoire où Kafka sera installé
- `PATH` : enrichi pour pouvoir appeler les scripts `kafka-*` directement

Sous Linux / macOS (Bash ou Zsh)

```
# Ajoutez ces lignes à la fin de ~/.bashrc (ou ~/.zshrc)
export KAFKA_HOME="$HOME/kafka"
```

```
export PATH="$KAFKA_HOME/bin:$PATH"

# Rechargez le shell
source ~/.bashrc # ou source ~/.zshrc
```

Sous Windows (PowerShell)

```
# À ajouter dans votre profil PowerShell ($PROFILE)
$env:KAFKA_HOME = "C:\kafka"
$env:Path = "$env:KAFKA_HOME\bin\windows;$env:Path"

# Pour le rendre permanent (à exécuter une fois en admin)
[Environment]::SetEnvironmentVariable("KAFKA_HOME", "C:\kafka", "User")
```

À retenir

Sur Windows, les scripts Kafka sont dans bin\windows\ et utilisent l'extension .bat (ex : kafka-topics.bat).

Sur Linux/macOS, ils sont dans bin/ et utilisent l'extension .sh (ex : kafka-topics.sh).

Dans la suite du TP, j'utilise les noms Linux ; les utilisateurs Windows remplaceront .sh par .bat.

Partie 2 — Installation de Kafka et premier démarrage

2.1 Téléchargement de Kafka

Nous utilisons la dernière version stable de Kafka. Au moment de la rédaction de ce TP, il s'agit de la version 3.9.x (version 3.x dernière, avant la transition complète à 4.x). La procédure reste identique pour toutes les versions 3.x récentes en mode KRaft.

Étape 1 — Récupération de l'archive

```
# Linux / macOS – placez-vous dans votre dossier home
cd ~

# Téléchargement (version Scala 2.13)
wget https://downloads.apache.org/kafka/3.9.0/kafka_2.13-3.9.0.tgz

# Décompression
tar -xzf kafka_2.13-3.9.0.tgz

# Renommage pour simplifier (cible de KAFKA_HOME)
mv kafka_2.13-3.9.0 kafka

# Vérification
ls $KAFKA_HOME/bin | head
```

Sous Windows

1. Téléchargez l'archive .tgz depuis <https://kafka.apache.org/downloads>
2. Extrayez-la avec 7-Zip à la racine du disque C:\
3. Renommez le dossier en C:\kafka
4. Évitez les chemins contenant des espaces ou caractères accentués (ex : pas dans Documents)

Étape 2 — Vérification de l'arborescence

```
tree -L 1 $KAFKA_HOME

# Sortie attendue
# /home/etudiant/kafka
# ├── bin          ← scripts CLI
# ├── config       ← fichiers de configuration
# ├── libs         ← dépendances Java
# ├── licenses
# ├── LICENSE
# ├── NOTICE
# └── site-docs
```

2.2 Configuration KRaft

Le mode KRaft utilise un fichier de configuration spécifique, fourni dans `config/kraft/server.properties`. Nous allons en faire une copie personnelle pour ne pas écraser le modèle officiel.

```
# Création d'un dossier pour les logs et un autre pour notre config
mkdir -p ~/kafka-data
cp $KAFKA_HOME/config/kraft/server.properties ~/kafka-data/server.properties
```

Éditez ensuite `~/kafka-data/server.properties` et modifiez ces lignes :

```
# Identifiant unique du nœud (entier >= 0)
node.id=1

# Le rôle : à la fois broker et controller (mode "combined" sur une seule machine)
process.roles=broker,controller

# Quorum des controllers (nous n'en avons qu'un)
controller.quorum.voters=1@localhost:9093

# Adresse d'écoute pour les clients (port 9092) et pour le contrôleur (port 9093)
listeners=PLAINTEXT://:9092,CONTROLLER://:9093

# Adresse annoncée aux clients (utilisée par les producers/consumers pour se reconnecter)
advertised.listeners=PLAINTEXT://localhost:9092

# Nom du listener pour les communications inter-controller
controller.listener.names=CONTROLLER

# Mapping des protocoles
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT

# Répertoire des logs (TRÈS important : évitez /tmp qui est nettoyé au reboot !)
log.dirs=/home/etudiant/kafka-data/logs

# Pour un environnement de TP – on autorise la création automatique de topics
auto.create.topics.enable=true

# Tolérance à 1 broker (sinon les topics internes refusent de démarrer)
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
```

Erreur fréquente : `log.dirs` dans `/tmp`

La configuration par défaut de Kafka pointe vers `/tmp/kraft-combined-logs`. Sous Linux, `/tmp` est généralement vidé au redémarrage. Si vous laissez cette valeur, votre cluster perdra tous les topics et messages au prochain reboot. Utilisez systématiquement un chemin sous votre HOME.

2.3 Formatage du stockage

Avant le tout premier démarrage, KRaft impose de formater le répertoire de logs avec un identifiant unique de cluster. Cette étape n'est faite qu'une seule fois ; ne la refaites pas après chaque arrêt, vous perdriez vos données.

```
# Génération d'un UUID de cluster unique
KAFKA_CLUSTER_ID=$(kafka-storage.sh random-uuid)
```

```
echo "Mon cluster ID : $KAFKA_CLUSTER_ID"

# Formatage du répertoire de logs avec cet ID
kafka-storage.sh format \
  --config ~/kafka-data/server.properties \
  --cluster-id $KAFKA_CLUSTER_ID

# Sortie attendue : "Formatting metadata directory ... with metadata.version 3.9-IV0"
```

2.4 Démarrage du broker

```
# Démarrage en avant-plan (pour voir les logs)
kafka-server-start.sh ~/kafka-data/server.properties

# Pour démarrer en arrière-plan
kafka-server-start.sh -daemon ~/kafka-data/server.properties
```

Si tout va bien, vous voyez défiler beaucoup de logs INFO et la phrase clé apparaît :

```
[KafkaServer id=1] started (kafka.server.KafkaServer)
```

Test de bonne santé

Dans un autre terminal, exécutez : `kafka-broker-api-versions.sh --bootstrap-server localhost:9092`
Si la commande renvoie la liste des API versions sans erreur, le broker est opérationnel.

2.5 Arrêt propre du broker

L'arrêt propre est important : Kafka doit pouvoir flusher ses buffers et libérer ses verrous. Ne tuez JAMAIS le processus avec `kill -9` sauf en dernier recours.

```
# Si le broker tourne en avant-plan : Ctrl+C dans son terminal

# Si le broker tourne en arrière-plan
kafka-server-stop.sh

# Vérification qu'aucun processus Kafka ne tourne
ps -ef | grep kafka | grep -v grep
```

Exercice 2.A — Premier démarrage

Énoncé

1. Installez Kafka selon la procédure ci-dessus.
2. Modifiez le fichier `server.properties` pour pointer `log.dirs` vers un dossier de votre HOME.
3. Formatez le stockage avec un `cluster-id` que vous noterez sur votre compte-rendu.
4. Démarrez le broker en avant-plan.
5. Dans un second terminal, listez les versions d'API supportées.
6. Arrêtez proprement le broker.

Livrable : capture d'écran du démarrage réussi + cluster-id généré.

Partie 3 — Manipulation en ligne de commande

Tous les exercices de cette partie supposent que votre broker tourne en arrière-plan. Démarrez-le avec `kafka-server-start.sh -daemon ~/kafka-data/server.properties` avant de continuer.

3.1 Gestion des topics

Création d'un topic

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --create \
  --topic ventes \
  --partitions 3 \
  --replication-factor 1
```

Trois **partitions** signifient que les messages publiés dans le topic `ventes` seront répartis sur trois fichiers logs distincts. Chaque partition garantit l'ordre interne des messages, mais il n'y a pas d'ordre global entre partitions. Le facteur de réplication est ici à 1 car nous n'avons qu'un seul broker — en production, on choisit toujours 3.

Lister les topics

```
# Liste simple
kafka-topics.sh --bootstrap-server localhost:9092 --list

# Détails complets
kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic ventes
```

Le `--describe` affiche pour chaque partition :

- Leader : le broker responsable des lectures/écritures
- Replicas : les brokers qui hébergent une copie
- Isr (In-Sync Replicas) : les copies à jour

Modifier les partitions

Le nombre de partitions d'un topic ne peut qu'augmenter, jamais diminuer. C'est une décision irréversible, à prendre avec soin (l'ajout casse le partitionnement par clé existant).

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --alter --topic ventes --partitions 5
```

Supprimer un topic

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --delete --topic ventes
```

La suppression est asynchrone

Lorsque vous supprimez un topic, Kafka marque les fichiers comme à supprimer puis les nettoie en arrière-plan (selon `log.cleaner.threads`). Il peut s'écouler quelques secondes avant que le topic

disparaissent vraiment de --list.

3.2 Producteur en console

Le script `kafka-console-producer.sh` permet d'envoyer des messages depuis le clavier. C'est l'outil parfait pour tester rapidement.

```
kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic ventes

# Tapez vos messages (un par ligne) puis Ctrl+D pour quitter
> Vente #1 – Tunis – 250 DT
> Vente #2 – Sousse – 480 DT
> Vente #3 – Sfax – 120 DT
```

Envoyer des messages avec clé

Par défaut, le producteur n'envoie pas de clé. Or, la clé détermine la partition de destination via un hachage : tous les messages avec la même clé arrivent dans la même partition, et donc dans le même ordre. C'est crucial pour préserver la cohérence (par exemple : tous les événements d'un même client).

```
kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic ventes \
  --property "parse.key=true" \
  --property "key.separator=:"

> client-A:Achat 250 DT
> client-B:Achat 480 DT
> client-A:Achat 90 DT    ← ira dans la MÊME partition que le premier "client-A"
```

3.3 Consommateur en console

Lecture depuis le début

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic ventes \
  --from-beginning
```

Lecture avec clé et métadonnées

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic ventes \
  --from-beginning \
  --property "print.key=true" \
  --property "print.partition=true" \
  --property "print.offset=true" \
  --property "key.separator= | "
```

Comportement par défaut sans --from-beginning

Sans cette option, kafka-console-consumer ne lit que les messages produits APRÈS son démarrage. Démarrez d'abord le consumer dans un terminal, puis le producer dans un second terminal : c'est la démo classique du flux temps réel.

3.4 Groupes de consommateurs

Quand plusieurs consommateurs lisent le même topic dans le même groupe, Kafka leur distribue les partitions : un même message n'est livré qu'à un seul membre du groupe. Cela permet la mise à l'échelle horizontale du traitement.

Démarrer un consumer dans un groupe nommé

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic ventes \
  --group analyse-ventes \
  --from-beginning
```

Lister et inspecter les groupes

```
# Lister tous les groupes
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list

# Décrire un groupe (affiche le LAG par partition)
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group analyse-ventes
```

La commande `--describe` affiche pour chaque partition consommée par le groupe :

Colonne	Signification
TOPIC	Topic concerné
PARTITION	Numéro de la partition
CURRENT-OFFSET	Position actuelle du consommateur
LOG-END-OFFSET	Dernière position écrite par les producteurs
LAG	Différence (LOG-END - CURRENT) — combien de messages reste-t-il à lire
CONSUMER-ID	Identifiant unique du consommateur
HOST	Adresse IP du client

Le LAG est l'indicateur n°1 à surveiller en production

Un LAG qui croît continuellement signifie que vos consommateurs n'arrivent pas à suivre le débit des producteurs. Solutions : ajouter des consommateurs dans le groupe (jusqu'à concurrence du nombre de partitions), augmenter le nombre de partitions, optimiser le code de traitement.

Réinitialiser les offsets

Cas d'usage classique : un bug a corrompu les calculs ; on veut rejouer les messages depuis le début. Le consommateur DOIT être arrêté avant cette opération.

```
# Rejouer depuis le début
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group analyse-ventes \
  --reset-offsets --to-earliest \
  --topic ventes --execute

# Sauter à la dernière position (ignorer le passé)
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group analyse-ventes \
  --reset-offsets --to-latest \
  --topic ventes --execute

# Reculer de 100 messages
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group analyse-ventes \
  --reset-offsets --shift-by -100 \
  --topic ventes --execute

# Aller à un timestamp précis (rejouer la dernière heure)
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group analyse-ventes \
  --reset-offsets --to-datetime 2026-04-28T10:00:00.000 \
  --topic ventes --execute
```

Exercice 3.A — Création et exploration

Énoncé

1. Créez un topic logs-application avec 4 partitions.
2. Décrivez le topic et identifiez le leader de chaque partition.
3. Avec kafka-console-producer, envoyez 10 messages SANS clé.
4. Avec un consumer dans le groupe analyseur-1, lisez tous les messages depuis le début, en affichant la partition.

Question : Les messages sont-ils répartis équitablement sur les 4 partitions ? Pourquoi ?

Exercice 3.B — Effet de la clé sur le partitionnement

Énoncé

1. Sur le même topic logs-application, envoyez maintenant 12 messages AVEC clé, en utilisant 3 clés différentes (ex : serveur-1, serveur-2, serveur-3) — soit 4 messages par clé.
2. Lisez tous les messages en affichant la clé, la partition et l'offset.

Question 1 : Tous les messages d'une même clé arrivent-ils dans la même partition ?

Question 2 : Que se passerait-il si vous augmentiez le nombre de partitions à 8 et envoyiez à nouveau des messages avec les mêmes clés ?

Exercice 3.C — Groupes de consommateurs et parallélisme

Énoncé

1. Démarrez DEUX consommateurs dans le même groupe groupe-A, lisant tous les deux le topic logs-application.
2. Dans un troisième terminal, démarrez un producteur et envoyez une centaine de messages.
3. Observez quels messages vont à quel consommateur.
4. Démarrez un troisième consommateur dans le même groupe.
5. Décrivez le groupe avec `--describe` pour observer le rebalance.

Question 1 : Combien de partitions chaque consommateur lit-il dans chaque configuration (2 puis 3 consommateurs) ?

Question 2 : Que se passe-t-il si vous démarrez un 5ème consommateur (alors que le topic n'a que 4 partitions) ?

Partie 4 — Programmation Producer / Consumer en Java

La CLI suffit pour les démos, mais en production on utilise les API Kafka depuis du code applicatif. Java est le langage de référence (le client Java est officiel et le plus complet), mais des clients existent pour quasiment tous les langages : Python (kafka-python, confluent-kafka), Go (sarama, franz-go), Node.js (kafkajs), .NET, etc.

4.1 Création du projet Maven

```
mvn archetype:generate \
  -DgroupId=tn.utm.kafka \
  -DartifactId=tp-kafka-java \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4 \
  -DinteractiveMode=false

cd tp-kafka-java
```

Fichier pom.xml

Remplacez le contenu de pom.xml par celui ci-dessous. La dépendance principale est kafka-clients ; nous ajoutons aussi slf4j pour avoir des logs propres.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>tn.utm.kafka</groupId>
  <artifactId>tp-kafka-java</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <kafka.version>3.9.0</kafka.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
      <version>${kafka.version}</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.13</version>
    </dependency>
  </dependencies>
</project>
```

4.2 Premier producteur

Créez le fichier `src/main/java/tn/utm/kafka/SimpleProducer.java` :

```
package tn.utm.kafka;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class SimpleProducer {

    public static void main(String[] args) {
        // Configuration du producteur
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        // Garanties de durabilité
        props.put(ProducerConfig.ACKS_CONFIG, "all");           // attend l'ack de tous
les replicas
        props.put(ProducerConfig.RETRIES_CONFIG, 3);           // 3 tentatives en cas
d'échec
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true); // évite les doublons

        try (Producer<String, String> producer = new KafkaProducer<>(props)) {
            for (int i = 1; i <= 10; i++) {
                String key = "client-" + (i % 3);
                String value = "Achat numéro " + i + " (" + (50 + i * 17) + " DT)";

                ProducerRecord<String, String> record =
                    new ProducerRecord<>("ventes", key, value);

                // Envoi asynchrone avec callback
                producer.send(record, (metadata, exception) -> {
                    if (exception != null) {
                        System.err.println("Échec d'envoi : " + exception.getMessage());
                    } else {
                        System.out.printf(
                            "Envoyé – partition=%d, offset=%d, key=%s\n",
                            metadata.partition(), metadata.offset(), key
                        );
                    }
                });
            }
            // flush() force l'envoi des messages bufferisés avant la fermeture
            producer.flush();
        }
    }
}
```

Compilation et exécution

```
mvn clean package
mvn exec:java -Dexec.mainClass="tn.utm.kafka.SimpleProducer"
```

4.3 Premier consommateur

Créez `src/main/java/tn/utm/kafka/SimpleConsumer.java` :


```

package tn.utm.kafka;

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class SimpleConsumer {

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "groupe-java-1");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());

        // earliest = depuis le début (si aucun offset commité), latest = à partir de
maintenant
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

        // Commit manuel (recommandé pour traitement fiable)
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);

        try (Consumer<String, String> consumer = new KafkaConsumer<>(props)) {
            consumer.subscribe(Collections.singletonList("ventes"));

            System.out.println("  En attente de messages... (Ctrl+C pour arrêter)");

            while (true) {
                ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(500));

                for (ConsumerRecord<String, String> record : records) {
                    System.out.printf(
                        "  partition=%d, offset=%d, key=%s, value=%s\n",
                        record.partition(), record.offset(), record.key(), record.value()
                    );
                    // Ici on simule le traitement métier
                    // ...
                }

                // Commit synchrone après traitement réussi
                if (!records.isEmpty()) {
                    consumer.commitSync();
                }
            }
        }
    }
}

```

Auto-commit vs commit manuel

Avec `ENABLE_AUTO_COMMIT_CONFIG = true` (défaut), Kafka commite automatiquement les offsets toutes les 5 secondes (`auto.commit.interval.ms`). C'est simple mais risqué : si le consommateur plante après avoir lu mais avant d'avoir traité, les messages sont considérés comme traités et perdus.

Avec le commit manuel (`commitSync` ou `commitAsync`), vous ne validez l'offset qu'après le

traitement réussi. C'est la pratique recommandée pour toute application où la perte de message est inacceptable.

4.4 Sérialisation d'objets métier

Les String c'est bien pour démarrer, mais en pratique on échange des objets structurés. Trois approches courantes :

1. JSON via Jackson — simple, lisible, mais verbeux
2. Apache Avro avec Schema Registry — compact, fortement typé, gère l'évolution de schéma
3. Protobuf — très compact, multi-langages

Exemple JSON minimaliste avec Jackson :

```
// Ajoutez à pom.xml
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.17.2</version>
</dependency>

package tn.utm.kafka;

public class Vente {
  private String idClient;
  private double montant;
  private String ville;

  public Vente() {} // requis par Jackson
  public Vente(String idClient, double montant, String ville) {
    this.idClient = idClient;
    this.montant = montant;
    this.ville = ville;
  }
  // getters / setters omis...
}

// Producer JSON
import com.fasterxml.jackson.databind.ObjectMapper;
// ...
ObjectMapper mapper = new ObjectMapper();
Vente vente = new Vente("client-007", 350.0, "Tunis");
String json = mapper.writeValueAsString(vente);

producer.send(new ProducerRecord<>("ventes", vente.getIdClient(), json));
```

Exercice 4.A — Producer / Consumer en boucle

Énoncé

1. Implémentez SimpleProducer et SimpleConsumer comme indiqué.
2. Lancez le consumer dans un terminal.
3. Lancez le producer dans un autre terminal.
4. Vérifiez que tous les messages sont consommés et que les offsets s'incrémentent correctement.

Question : Si vous lancez DEUX instances du consumer (dans le même groupe-java-1), comment se répartit la charge ?

Exercice 4.B — Idempotence et fiabilité

Énoncé

Désactivez `ENABLE_IDEMPOTENCE_CONFIG` dans le producteur. Lancez le producteur, puis pendant l'exécution, arrêtez brutalement le broker (kill -9) et redémarrez-le rapidement.

1. Que se passe-t-il côté producteur ?
2. Réactivez l'idempotence et refaites le test : quelle différence observez-vous ?
3. Expliquez le rôle des numéros de séquence dans le protocole d'idempotence (cf. KIP-98).

Exercice 4.C — Sérialisation JSON

Énoncé

1. Créez une classe métier `Commande` avec au moins 4 champs (id, date, articles, total).
2. Modifiez le producteur pour envoyer des `Commande` sérialisées en JSON.
3. Modifiez le consommateur pour désérialiser et afficher proprement chaque commande.
4. Bonus : créez un sérialiseur Kafka custom (`CommandeSerializer` implements `Serializer<Commande>`) au lieu de gérer le mapping JSON dans le producer.

Partie 5 — Partitions, offsets, garanties de livraison

5.1 Le partitionnement en détail

Le partitionnement est ce qui rend Kafka scalable. Plus un topic a de partitions, plus on peut paralléliser à la fois la production et la consommation. Mais ce parallélisme a un coût et des contraintes.

Comment Kafka choisit-il une partition ?

4. Si une partition est explicitement spécifiée dans le `ProducerRecord` → elle est utilisée.
5. Sinon, si une clé est présente → $\text{hash}(\text{key}) \% \text{nombre_de_partitions}$.
6. Sinon (clé null), depuis Kafka 2.4+ → sticky partitioner : un batch entier va à la même partition pour optimiser le débit.

Conséquence pratique

Si vous augmentez le nombre de partitions d'un topic existant, le mapping clé→partition CHANGE. Les nouveaux messages d'une clé donnée peuvent atterrir dans une partition différente, brisant la garantie d'ordre. C'est pourquoi on choisit dès le départ un nombre de partitions un peu surdimensionné.

Combien de partitions choisir ?

Une heuristique répandue : $\text{nombre de partitions} = \max(\text{débit_souhaité} / \text{débit_par_partition_consumer}, \text{débit_souhaité} / \text{débit_par_partition_producer})$. En pratique, pour un topic actif :

- Trop peu (1-2) : pas de parallélisme, latence d'attente.
- Bon ordre de grandeur : entre 6 et 30 partitions par topic.
- Trop (centaines/milliers) : pression sur le contrôleur, ouverture de fichiers, mémoire de buffers.

5.2 Sémantiques de livraison

Sémantique	Garantie	Comment l'obtenir
At most once	0 ou 1 fois (perte possible)	auto-commit avant traitement
At least once	1 ou plusieurs fois (doublons possibles)	commit manuel APRÈS traitement
Exactly once	exactement 1 fois	transactions Kafka + idempotence

Pour la grande majorité des cas, on vise at least once + idempotence applicative (le consumer doit pouvoir traiter deux fois le même message sans effet de bord — par exemple en utilisant l'id du message comme clé primaire).

Exactly-once : transactions Kafka

Pour les pipelines critiques où la duplication est inacceptable (ex : virements bancaires), Kafka propose les transactions. Le pattern read-process-write : on lit d'un topic, on transforme, on écrit dans un autre, et l'ensemble est atomique.

```
Properties props = new Properties();
// ... config standard ...
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "mon-pipeline-tx-1");
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
producer.initTransactions();

try {
    producer.beginTransaction();

    for (ConsumerRecord<String, String> rec : recordsLus) {
        // Transformation
        String resultat = transformer(rec.value());
        producer.send(new ProducerRecord<>("topic-sortie", rec.key(), resultat));
    }
    // Commit des offsets DANS la transaction
    producer.sendOffsetsToTransaction(offsetsAcommiter, "groupe-consommateur");
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
    throw e;
}
```

5.3 Rétenion et compactage

Kafka conserve les messages pendant une durée configurable. Deux politiques :

Rétention par durée (delete)

```
# Conserver 7 jours
kafka-configs.sh --bootstrap-server localhost:9092 \
  --entity-type topics --entity-name ventes \
  --alter --add-config retention.ms=604800000

# Conserver max 1 Go par partition
kafka-configs.sh --bootstrap-server localhost:9092 \
  --entity-type topics --entity-name ventes \
  --alter --add-config retention.bytes=1073741824
```

Compactage (compact)

Avec cleanup.policy=compact, Kafka conserve indéfiniment la dernière valeur connue pour chaque clé, et purge les anciennes versions. Idéal pour stocker un état (ex : profil utilisateur courant).

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic profils-utilisateurs \
  --partitions 3 --replication-factor 1 \
  --config cleanup.policy=compact
```

Effacer une clé en compact

Pour supprimer une clé d'un topic compacté, on envoie un message avec cette clé et un payload null. C'est ce qu'on appelle un tombstone. Après la prochaine compaction, la clé disparaît du topic.

Exercice 5.A — Mesurer le LAG sous charge

Énoncé

1. Créez un topic stress-test à 6 partitions.
2. Avec kafka-producer-perf-test, envoyez 1 million de messages de 100 octets à 10 000 messages/seconde :

```
kafka-producer-perf-test.sh --topic stress-test \
  --num-records 1000000 --record-size 100 \
  --throughput 10000 \
  --producer-props bootstrap.servers=localhost:9092
```

3. Pendant l'envoi, démarrez un seul consumer dans un groupe perf-1.
4. Toutes les 10 secondes, exécutez kafka-consumer-groups --describe --group perf-1.

Question : Le LAG augmente-t-il, diminue-t-il, ou reste-t-il stable ? Que se passe-t-il si vous démarrez 5 consumers supplémentaires dans le même groupe ?

Exercice 5.B — Topic compacté

Énoncé

1. Créez un topic compact appelé etat-utilisateurs.
2. Envoyez plusieurs versions du même utilisateur :


```
user-42 : { 'nom': 'Heithem', 'pays': 'TN' }
user-42 : { 'nom': 'Heithem', 'pays': 'TN', 'ville': 'Tunis' }
user-42 : { 'nom': 'Heithem', 'pays': 'TN', 'ville': 'Tunis', 'role': 'admin' }
```
3. Lisez le topic depuis le début après quelques minutes.
4. Envoyez un tombstone : user-42 → null.
5. Lisez à nouveau.

Question : À quel moment exactement la compaction se déclenche-t-elle ? (cf. log.cleaner.min.cleanable.ratio)



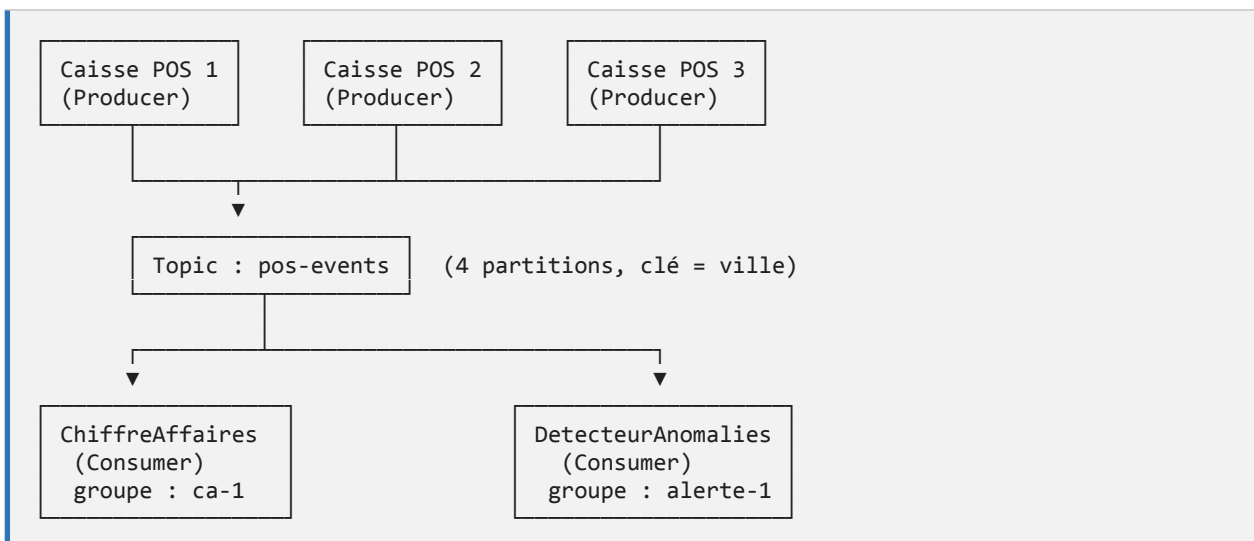
Partie 6 — Mini-projet : pipeline de logs en temps réel

6.1 Contexte fonctionnel

Vous travaillez pour une chaîne de magasins et devez agréger en temps réel les événements de logs venant de plusieurs caisses (POS) réparties sur plusieurs villes. Chaque caisse émet trois types d'événements : VENTE, RETOUR, OUVERTURE. L'objectif est :

7. Centraliser tous les événements dans un topic unique.
8. Calculer en continu le chiffre d'affaires par ville.
9. Détecter les retours anormaux (montant > 200 DT).

6.2 Architecture proposée



6.3 Format des messages

Chaque message est un JSON :

```
{
  "type": "VENTE",
  "idCaisse": "CAISSE-TUNIS-03",
  "ville": "Tunis",
  "timestamp": "2026-04-28T14:35:12.402Z",
  "montant": 175.50,
  "produits": ["pain", "lait", "fromage"]
}
```

6.4 Travail demandé

Étape 1 — Préparation

10. Créez le topic pos-events avec 4 partitions, replication-factor 1.

11. Le partitionnement se fait sur la clé = ville (les événements d'une même ville restent ordonnés).

Étape 2 — Producteur de simulation

Écrivez une classe `SimulateurCaisse` qui :

- Émet en continu (1 message toutes les 100 à 500 ms aléatoirement)
- Utilise une des villes parmi : Tunis, Sousse, Sfax, Bizerte, Gabès
- Choisit aléatoirement un type parmi VENTE (70%), RETOUR (10%), OUVERTURE (20%)
- Pour les VENTE et RETOUR, génère un montant aléatoire entre 5 et 500 DT
- Configure `ENABLE_IDEMPOTENCE_CONFIG=true` et `acks=all`

Étape 3 — Consommateur ChiffreAffaires

Écrivez `ChiffreAffairesParVille` qui :

- Maintient en mémoire une `Map<String, Double>` ville → CA cumulé (en ne comptant que les VENTE moins les RETOUR)
- Affiche le total par ville toutes les 5 secondes
- Commit manuel des offsets après traitement de chaque batch

Étape 4 — Consommateur DetecteurAnomalies

Écrivez `DetecteurAnomalies` qui :

- Lit le même topic dans un groupe DIFFÉRENT (alerte-1)
- Pour chaque RETOUR > 200 DT, écrit dans un nouveau topic alertes-retours
- Le topic d'alerte sert ensuite pour un éventuel système de notification (email/SMS)

Étape 5 — Tests à réaliser

12. Démarrez 1 simulateur, 1 ChiffreAffaires, 1 DetecteurAnomalies. Tout doit fonctionner.
13. Démarrez 2 simulateurs en parallèle (ils représentent 2 caisses).
14. Démarrez 3 instances de ChiffreAffaires dans le même groupe : observez le rebalance et la répartition des partitions.
15. Tuez une instance en plein traitement. Vérifiez que les messages non commités sont rejoués par les autres.
16. Faites un `--describe` sur le groupe `ca-1` toutes les 10 secondes pour observer le LAG en conditions réelles.

6.5 Livrables attendus

- Code source complet (projet Maven sur Git)
- Fichier `README.md` décrivant la procédure de démarrage

- Rapport PDF de 8 à 15 pages comportant : architecture, choix techniques, captures d'écran des tests, analyse du comportement de Kafka pendant les rebalances, conclusion sur les limites du déploiement mono-broker
- Démonstration en séance (15 min par binôme)

Annexe A — Dépannage des problèmes courants

Le broker ne démarre pas : « `UnsupportedVersionException` »

Vous avez probablement formaté avec un `cluster.id` puis modifié le `node.id` ou inversé `broker/controller`. Solution : effacer `log.dirs` et reformater (attention, vous perdez tout).

```
rm -rf ~/kafka-data/logs
KAFKA_CLUSTER_ID=$(kafka-storage.sh random-uuid)
kafka-storage.sh format -c ~/kafka-data/server.properties -t $KAFKA_CLUSTER_ID
```

« `org.apache.kafka.common.errors.TimeoutException` » côté producer

Le producer n'arrive pas à joindre le broker. Vérifiez :

- Le broker tourne (jps doit afficher Kafka)
- Le port 9092 est ouvert : `netstat -tlnp | grep 9092` sous Linux
- `advertised.listeners` pointe sur `localhost` (pas sur une IP externe)
- Aucun pare-feu ne bloque le port

Le consumer ne consomme rien

- `auto.offset.reset` par défaut est `latest` : sans offset précédent, il ignore l'historique. Mettez `earliest` pour les tests.
- Avec un nouveau `group.id`, Kafka démarre depuis `latest`. Si vous lancez le consumer APRÈS le producer, vous ne voyez rien — relancez le producer.
- Vérifiez avec `kafka-consumer-groups` que votre groupe existe et a un assignment.

« `Number of insync replicas (1) is less than required (2)` »

Vous tentez de produire avec `acks=all` sur un topic en `replication-factor=1` et `min.insync.replicas=2`. Soit vous baissez `min.insync.replicas` à 1, soit vous augmentez le nombre de brokers.

Disk full / `log.dirs` saturé

Réduisez la rétention ou nettoyez :

```
# Réduire la rétention à 1 jour
kafka-configs.sh --bootstrap-server localhost:9092 \
  --entity-type topics --entity-name MON_TOPIC \
  --alter --add-config retention.ms=86400000

# Forcer la suppression des anciens segments en réduisant retention.bytes
```

Annexe B — Commandes utiles

Commande

Usage

<code>kafka-topics.sh --list</code>	Liste les topics
<code>kafka-topics.sh --describe --topic X</code>	Détail d'un topic
<code>kafka-console-producer.sh --topic X</code>	Envoi interactif
<code>kafka-console-consumer.sh --topic X --from-beginning</code>	Lecture depuis le début
<code>kafka-consumer-groups.sh --list</code>	Liste les groupes
<code>kafka-consumer-groups.sh --describe --group G</code>	Détail + LAG d'un groupe
<code>kafka-configs.sh --describe --entity-type topics --entity-name X</code>	Voir la configuration d'un topic
<code>kafka-producer-perf-test.sh</code>	Benchmark côté producteur
<code>kafka-consumer-perf-test.sh</code>	Benchmark côté consommateur
<code>kafka-log-dirs.sh --describe</code>	Taille des partitions sur disque
<code>kafka-dump-log.sh --files X.log</code>	Lecture brute d'un segment de log